

Senior C# .NET Developer (Backend)

Technical Vetting Assignment

Instructions & Requirements

Take Home Assignment: Cash Movements Service with Modern gRPC (Optional Legacy Gateway)

Context

You're joining a team modernizing an 18+ year-old financial administration platform. Parts of the system are being rewritten while legacy components continue to run in parallel. Operations staff record cash movements (deposits, withdrawals, transfers), view balances per account, and export large statements.

You will build a focused slice that persists movements to SQL Server, exposes a modern gRPC API for new consumers, and optionally provides a minimal legacy WCF NetTcp gateway so existing tools can call into the same behavior. The system must avoid double-counting when the same reference is submitted more than once, including when submissions arrive at the same time.

Core technologies to use:

- C# and .NET 8 for the new service layer (gRPC + protobuf)
- SQL Server for persistence
- .NET Framework 4.7.2+ (or 4.8) for an optional legacy gateway (WCF with NetTcp)

Focus of evaluation:

- C#/.NET backend service design and API contracts
- SQL Server data modeling and correctness under duplicate and concurrent submissions
- RPC surface area via modern gRPC (streaming for large results); WCF gateway is optional for bonus signal

Assumptions and developer environment:

- Use SQL Server LocalDB or a containerized SQL Server instance; include scripts to create schema and seed data.
- Use Visual Studio 2022 or the .NET SDK/CLI. Provide instructions to run the gRPC host (and, if implemented, the WCF gateway) concurrently on your machine.

Requirements

Business scenario

- Movement: a single cash movement for an account with fields:
 - externalRef: example MOV-20240715-000123
 - accountId: example ACC-001
 - currency: example ZAR
 - amount: example 12500.00 (positive for deposit, negative for withdrawal)
 - occurredAt: example 2024-07-15T10:42:31Z
 - narration: example Initial deposit
- Balance: the sum of movements for an account (you decide any additional fields).
- Statement: an ordered list of movements in a date range with a running total.

Constraints and notes

- Volume: a statement for a busy account can include 50,000+ rows.

- Uniqueness: externalRef must be unique per account. Re-submissions of the same pair (accountId, externalRef) must not double-count.
- Concurrency: submissions may arrive at the same time from multiple callers.
- Field conflicts: if a repeated externalRef for the same account differs from the original fields (for example, amount or currency), do not double-count; choose how to surface this and explain your choice in [SOLUTION.md](#).
- New and legacy entry points (if implemented) should produce consistent outcomes over the same data.
- Make reasonable simplifications where necessary; call them out in [SOLUTION.md](#).

Task 1: Domain and persistence

Implement the core domain and storage for cash movements and balances.

- Persist movements in SQL Server and define the tables/keys you need.
- Provide a way to record a movement and compute/get the current balance for an account.
- Ensure re-submitting the same (accountId, externalRef) does not inflate balances, including when submissions happen at the same time.
- Add automated tests for a few focused cases: duplicate handling under concurrency and balance math.
- Optional: include structured logs at key points you consider useful.

Task 2: Modern RPC: gRPC service (on .NET 8)

Expose functionality for new consumers using gRPC with protobuf.

- Define protobuf messages and service methods of your choosing.
- Support:
 - Submitting a movement
 - Retrieving current balance for an account
 - Exporting a statement for a given account and date range
- Handle large statement exports in a way that does not require the client or server to hold the entire result in memory at once.
- Choose an error approach you consider appropriate and explain your choices in [SOLUTION.md](#).

Task 3 (Optional): Legacy gateway: WCF (NetTcp) on .NET Framework

Provide a minimal legacy gateway that calls into the same backend.

- Expose one operation that returns the current balance for an account.
- You may choose any hosting model that runs locally and can operate alongside your gRPC host.
- Include run instructions and any configuration needed.

Optional extensions (only if time allows)

- Batch submission of movements from a client in a single request.
- Statement “export” to a file format of your choice initiated via the API.
- A small client utility (desktop or console) that consumes the large statement export and shows progress.

Deliverables

1. A single GitHub repository containing:

- All the source code for the assignment
- A [README.md](#) with instructions to run locally.
- A [SOLUTION.md](#) describing the design and trade-offs you made.

2. Video Demo (5-10 minutes using Loom or similar):

- Demonstrate the working project by doing a voice over
- Walk through your code architecture
- Explain key design decisions and any trade-offs made
- Highlight interesting technical implementations

Time Estimate

This should take ± 4 hours to complete, but you can take as long as you need.